Optimizing Croc Embedded-Class SoC Report

Your Repo URL
> https://github.com/ABKCourses/e260c-sp26-mp2-klfa

Additional Note: Our PPA results were based on Docker container provided by Croc. In a similar manner, the scripts we made use pathways that are based on the Croc container (these can easily be modified to be the pathways corresponding to the ece260c workspace container).

Optimizations

Your Optimizations

1. Optimization 2 - The number of SRAM words in the core and the aspect ratio of the floor plan was successfully swept for 3 variations showing PPA improvements.
2. Optimization 3 - Using vlsiffra, optimized adder and multipliers for different algorithms: hancarlson, brent-kung, and koggestone were successfully integrated into the core replacing unsigned multiplier, adder, and branch target address calculation resulting in PPA improvements.

Optimization 2

Concept & Implementation

For optimization 2, we parameterized chip height and chip width variables in the floorplan to control aspect ratio, and parameterized the number of SRAM words which is defined in the RTL code. To choose the best configuration for these parameters, we swept a range of values.
The variations were tried for aspect ratios were as follows (CHIP_W, CHIP_H, ASPECT RATIO): (1850, 2230, 0.832) , (2000,1830, 1.09) , (2230, 1850, 1.21) along with the baseline (1916, 1916, 1.0). A few other configurations were discarded because they prevented the padring from being placed. For the SRAM word variations, the core configuration was changed to try variations which include 256, 1024, 2048 as well as the default value: 512. To ensure that the floorplan would actually place the change of the modified SRAMs, the hardcoded SRAM

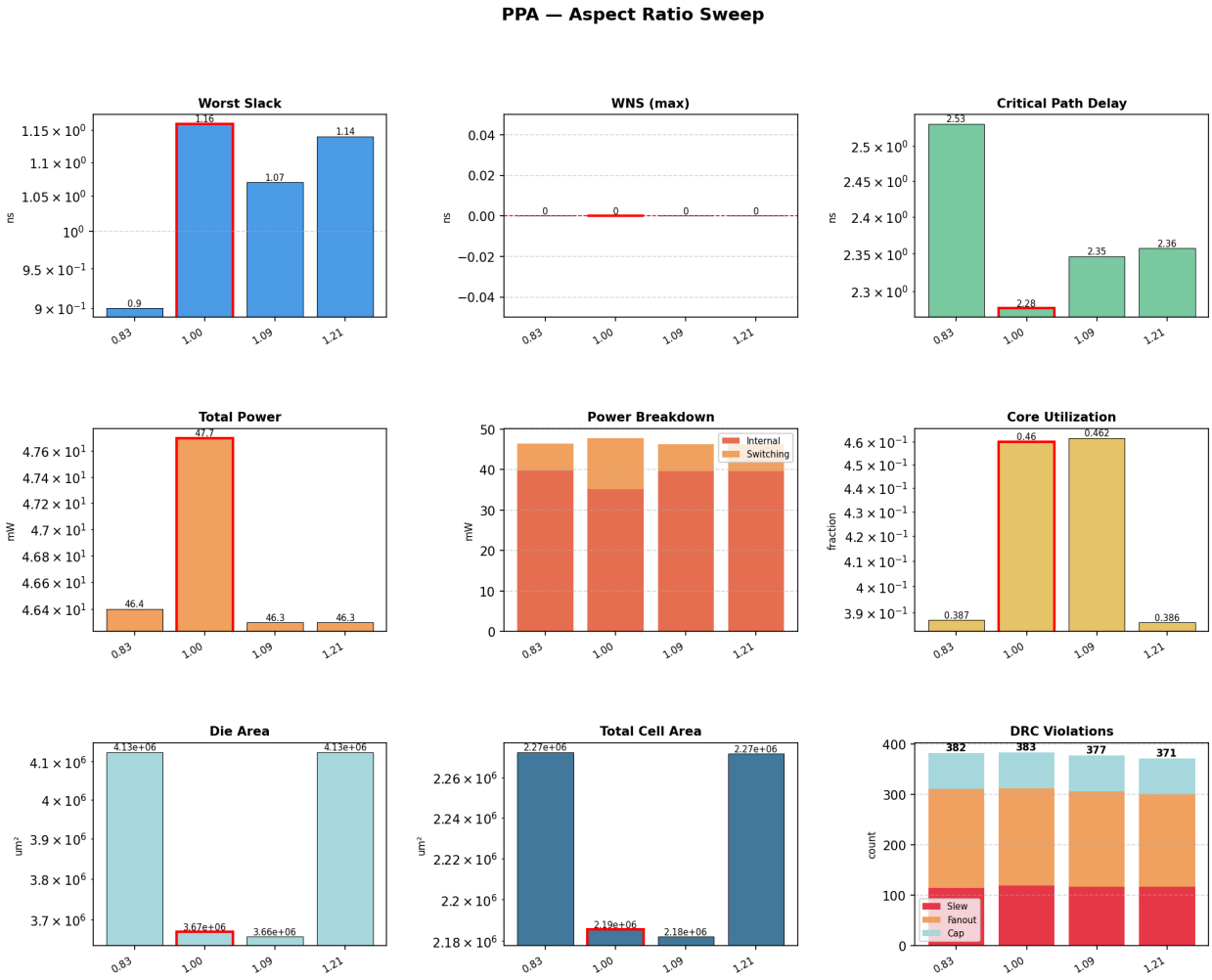
variables in the “floorplan” and “instances” tcl scripts were parameterized to ensure the rtl change was reflected in the design. The sweep results showed improvements in some PPA metrics while revealing tradeoffs for other metrics.

Sources & Tools

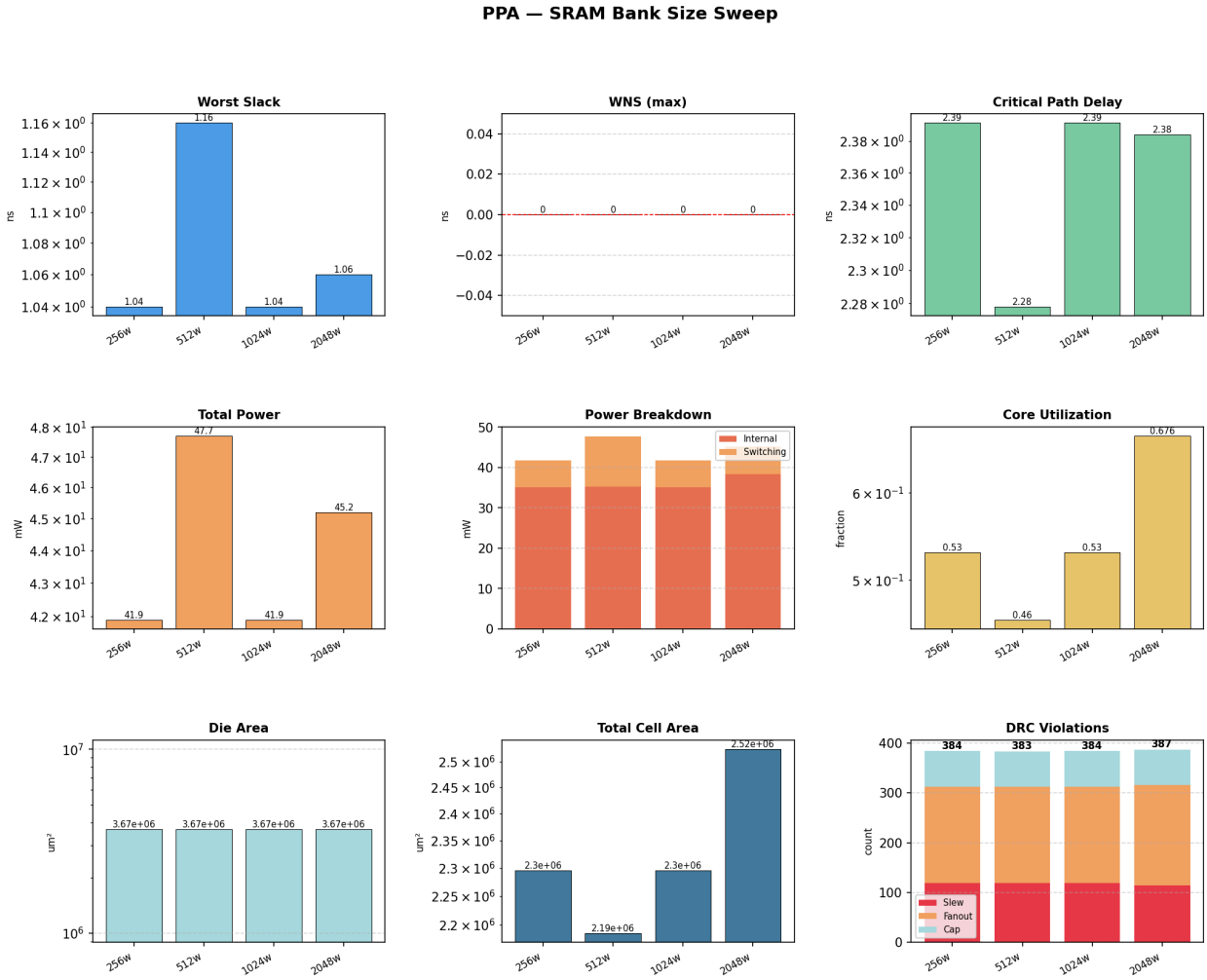
List the major scripts you added or changed in the repo to make this optimization possible. Then, list all additional tools, if any, that you use (e.g., VLSIFFRA). In 2-3 sentences each, describe the role each script/tool plays and what modifications, if any, you made.

The first script we developed was sweep_design_parameters.py which performs design space exploration on aspect ratio and number of SRAM words. The script sets environment variables for chip height, chip width, and number of sram words, and directly edits rtl config in the file, core_pkg.sv. The floorplan.tcl and instances.tcl scripts were changed to be parameterized to use the environment variables. The script then recompiles the rtl and runs the full backend flow and places the results in a folder, dse_results. Moreover, another script parse_sweep_results.py combs through the results and generates bar graphs for the following PPA metrics, worst slack, wns, critical path delay, power, power breakdown, core utilization, die area, total cell area, and drc violations.

Results & Data



Sweeping the aspect ratio showed no negative impact of wns or drc violations. Actually the 1.09 and 1.21 aspect ratios exhibit a reduction in the number of drc violations (377 and 371 vs default of 380). In terms of total power, all three variations showed much less power than default baseline(47.7). In particular the aspect ratios 1.09 and 1.21 had the least power(46.3). As such the portion of power that was switching decreased for all three variations. In terms of die and cell area, aspect ratios of 1.21 and 0.83 had increases in area while the aspect ratio of 1.09 actually produces less area (2.18e 06 um^2) than the default baseline(2.19e 06 um^2). In terms of timing, none of the variations improved upon the baseline with negative impact on critical path delay and worst slack (info: worst slack is positive, so higher “worst slack” is better performance). The aspect ratios of 0.83 and 1.21 have a much smaller core utilization than the baseline, but the aspect ratio of 1.09 has a slightly higher core utilization value of 0.467 compared to the baseline core utiliazation(0.46). From sweeping, it appears an aspect ratio of 1.09 provides the strongest benefits with decreased area, drc violations, and power.



Sweeping the number of SRAM words showed no negative impact of wns or drc violations. For power, all variations showed a reduction in power compared to the default baseline(47.7) with 256 and 1024 SRAM word sizes showing the strongest decrease(41.9). In terms of critical path delay, none of the variations improve upon the baseline and the same is the case for worst slack. In terms of die area, all variations match the baseline while for total cell area, all variations of SRAM word size show higher cell area. All SRAM variations showed a higher core utilization compared to the baseline. In terms of PPA impact, a SRAM word size of 1024 is optimal primarily for its large reduction in power and lesser increase in area than 2048 words.

Optimization 3

Concept & Implementation

For optimization 3, we used the vlsiffra repo to generate optimized adders and multipliers. The initial challenge was that the vlsiffra did not support the technology used by Croc SoC, IHP 130nm (ihp13). To address this, we had to create a python file that used ihp13 cells to make the required logic gates for the adders and multipliers: inverter, xor, and, half adder, full adder, 2-input AND into first input of 2-input OR, 2-input AND into both inputs of 2-input OR, and 3-input AND into both inputs of 2-input OR. Moreover, a section of the verilog file defining the ihp13 cells had to be provided to the vlsiffra. After completing these changes, we were able to generate optimized adders and multipliers for the following algorithms: Han Carlson, Brent-kung, and Koggestone. In the rtl for the Croc, the alu unit's adder and the branch target address calculation was replaced with vlsiffra generated adder. For the core's multiplier, the fast multiplier's unsigned multiplication logic was replaced with the vlsiffra generated multiplier. Only the unsigned multiplier was replaced because the vlsiffra generated multiplier cannot handle signed multiplication. Moreover, wrapping sign handling would add complexity and potentially hurt timing. Initially, we swept using Han Carlson to see the effect of just replacing the adder, just replacing the multiplier, just replacing branch target address calculations, and combinations of these replacements. After finding the optimal changes, we swept for the various algorithms, Han Carlson, Brent-kung, and Koggestone. These sweeps manifested in tangible improvements on PPA, particularly critical path delay.

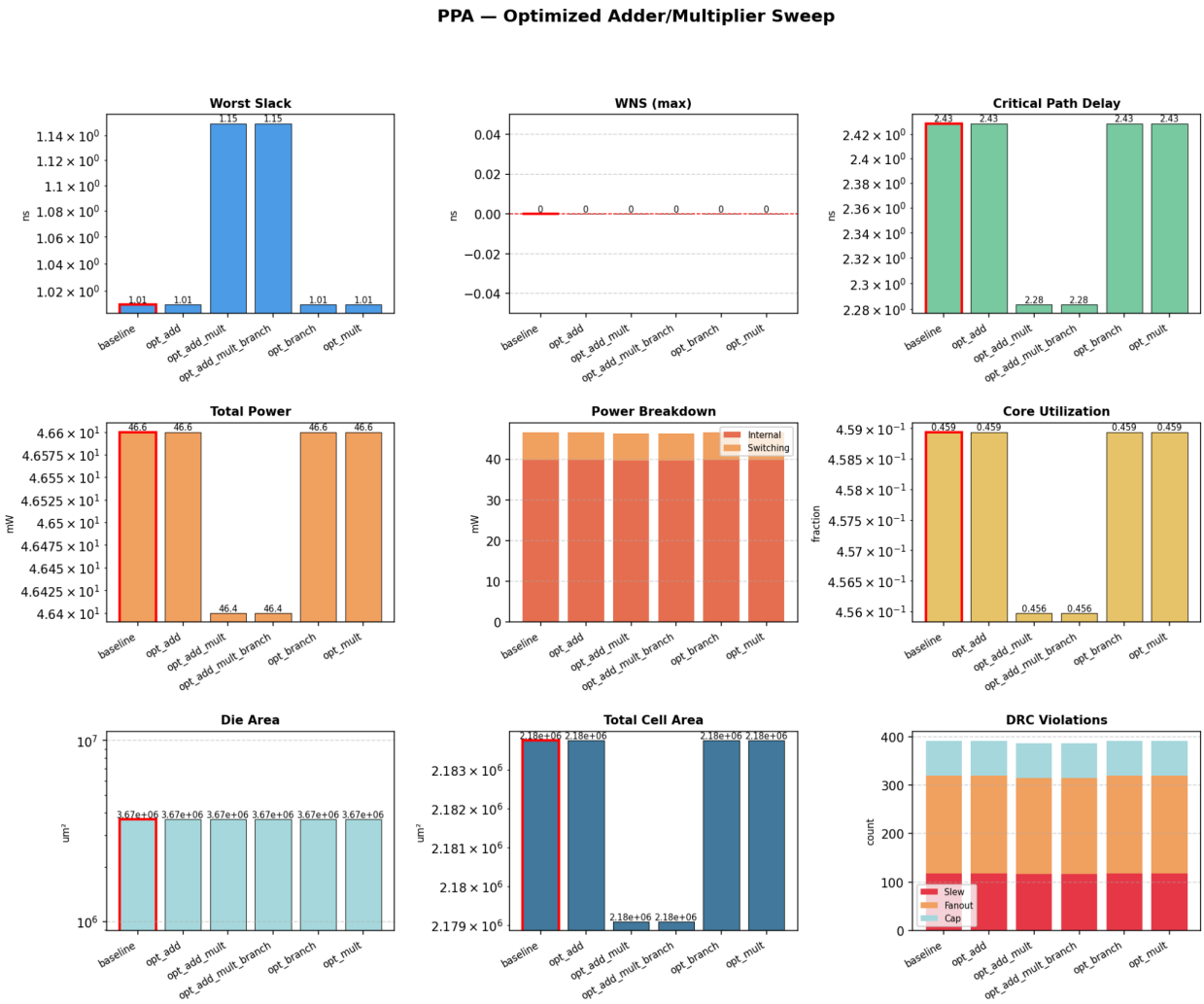
Sources & Tools

List the major scripts you added or changed in the repo to make this optimization possible. Then, list all additional tools, if any, that you use (e.g., VLSIFFRA). In 2-3 sentences each, describe the role each script/tool plays and what modifications, if any, you made.

The biggest changes were providing technology support for ihp13 within the vlsiffra submodule. A python file ihp13.py (inside the vlsiffra verilog folder) was created to create the necessary logic gates using ihp13 technology cells to generate the adder and multiplier. Moreover, a truncated verilog file was created using the ihp13 cells used for the file above. In the croc core cve2, the cve2_alu.sv, cve2_multdiv_fast.sv, and cve2_branch_predict.sv were changed to replace the corresponding logic with generated adder and multiplier. The sweeps for this part were performed manually using the commands in the readme and the results were stored in two places opt3_results and alg_sweep. Opt3_results contains the results from sweep using only hancarlson and looks at how incremental changes, only replacing adder or only replacing multiplier, affects the PPA metrics. Alg_sweep contains results from sweeping the various algorithms: han carlson, brent-kung, and koggestone. Lastly, the same script used

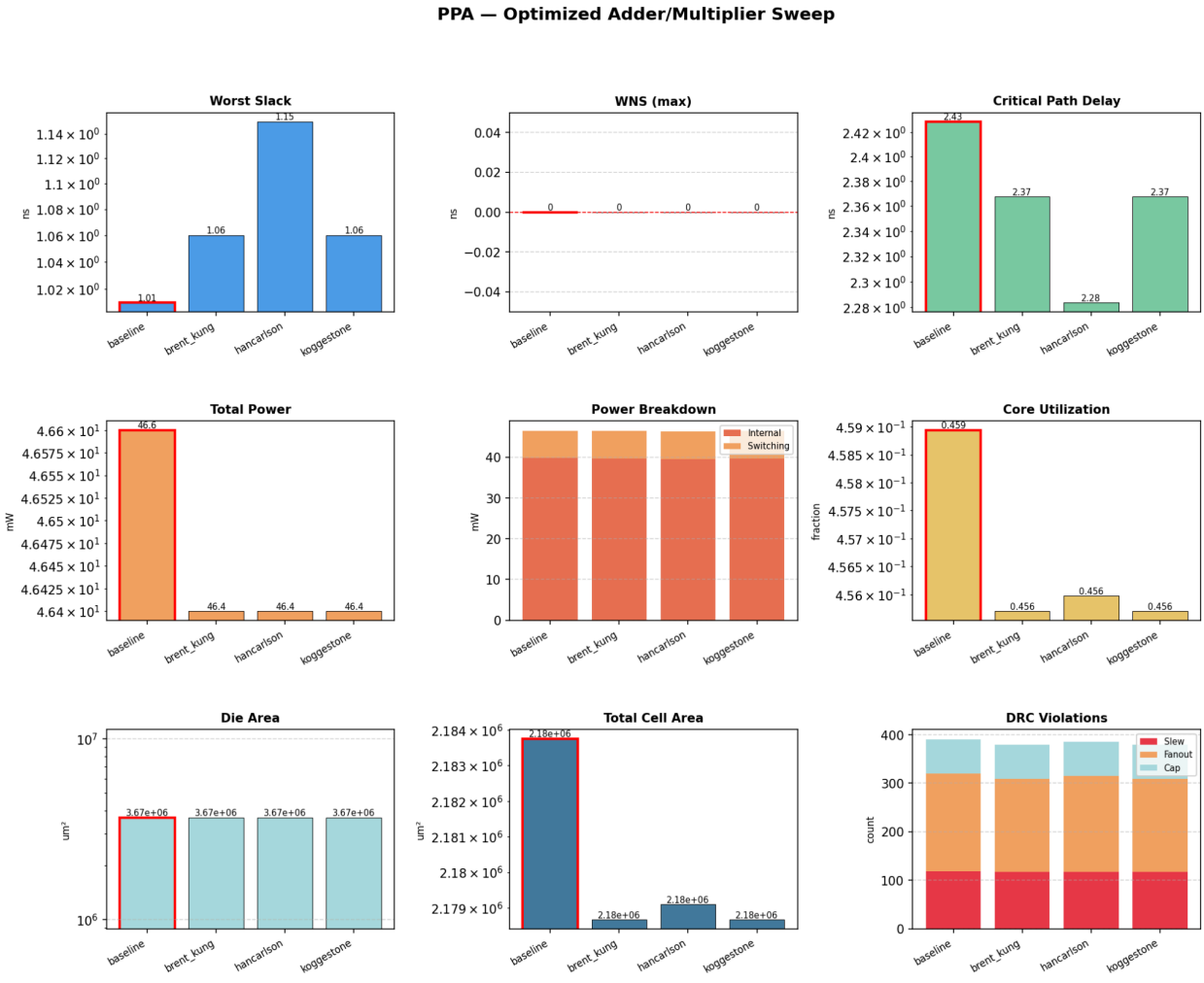
to parse the results of the previous optimization was used to parse the results of this sweep and generate graphs.

Results & Data



This shows the results of the sweep over the incremental changes in replacing various paths of core with optimized adders and multipliers from vlsiffra. The variations tested were only changing the alu adder(opt_add), only changing the unsigned multiplier in fast_multiply_div(opt_mult), only changing the branch target address calculator(opt_branch), changing both the alu and multiplier(opt_add_mult), and changing all three paths(opt_add_branch_mult). There is no negative impact on wns and die area. In terms of DRC violations, there is actually a slight decrease with opt_add_mult and

opt_add_mult_branch variations. The biggest gain is in critical path delay with opt_add_mult and opt_add_mult_branch having a much smaller delay(2.28) compared to the baseline(2.43). Likewise, there is a similar gain in worst slack with opt_add_mult and opt_add_mult_branch having a higher positive slack(1.15) compared to the baseline(1.01). Moreover, there is a reduction in power(46.4) with opt_add_mult and opt_add_mult_branch compared to the baseline(46.6). Similarly, there is a decrease in total cell area for opt_add_mult and opt_add_mult_branch compared to the baseline. In core utilization, these two variations also show a much smaller utilization than the baseline. As such, replacing these three paths in core for opt_add_branch_mult (replacing alu adder, unsigned multiplier, and branch target address calculation) seems to be the best design leading to tangible improvements in critical path delay, worst total slack, power, cell area, drc violations, and core utilization.



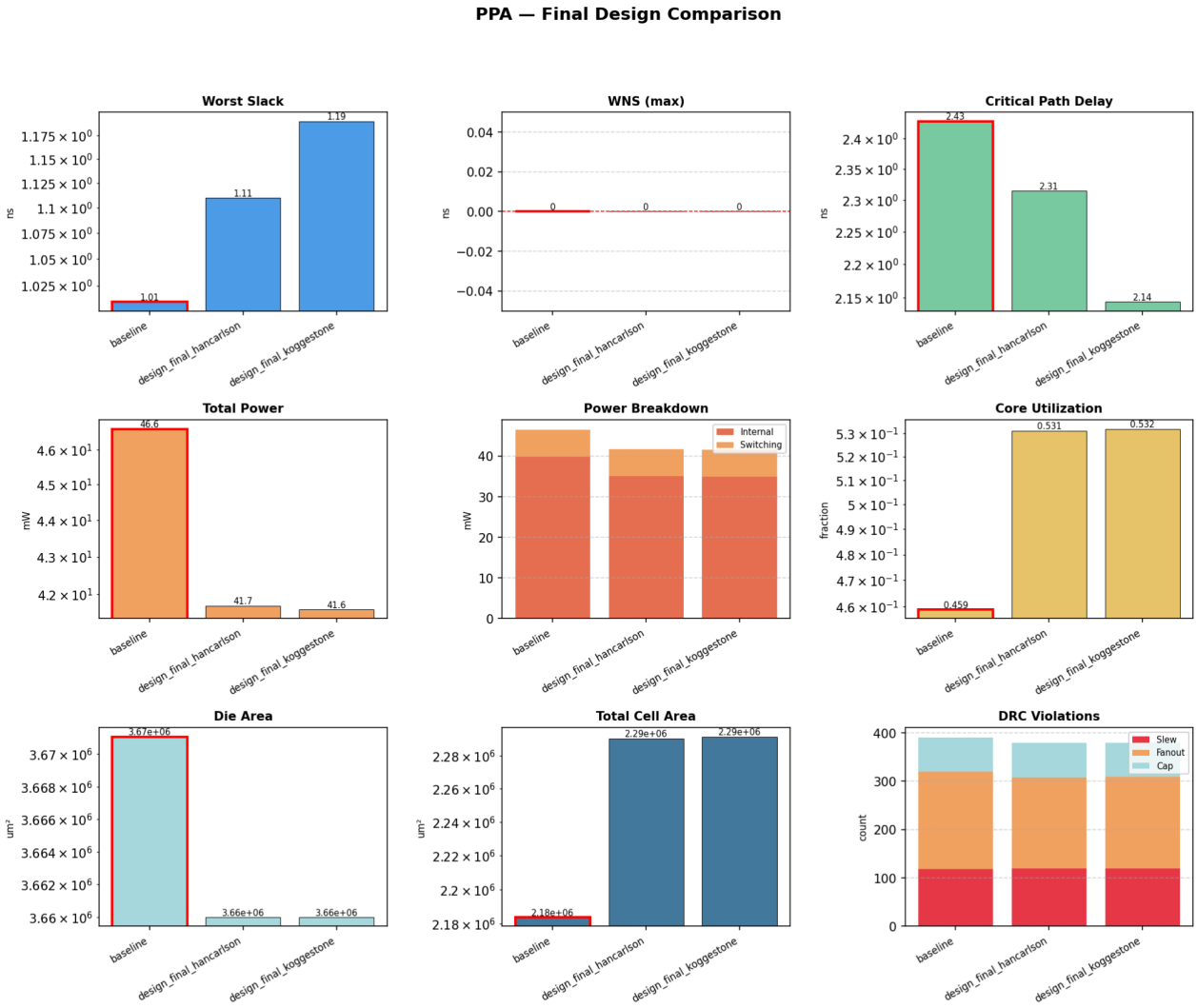
This sweep examines performance of the various algorithms(hancarlson, brent-kung, koggestone) that vlsiffra can use to generate adders and multipliers and uses the optimal

variation from the previous sweep(replacing alu adder, unsigned multiplier, and branch target address calculation). In terms of die area and wns, there is no impact. For timing, all three algorithms show decreases in critical path delay and correspondingly increases in worst slack. However, han carlson shows biggest improvement upon the baseline with critical path delay of 2.28 vs the baseline's 2.43 and worst slack of 1.15 vs the baseline's 1.06. In terms of power, all three algorithms show an identical reduction of power(46.4) compared to the baseline(46.6). In terms of total cell area, all three algorithms show decreases, but the brent-kung and koggestone algorithms both show the largest decrease with a cell area of 2178668.1408 um^2 compared to the baseline total cell area of 2183752.0896 um^2. Similarly, in terms of core utilization, both brent-kung and koggestone(0.4557) show a larger decrease than han carlson(0.455975) compared to the baseline(0.459). In terms of drc violations, brent-kung and koggestone show lower violations(380) than han carlson(386) which already has lower violations than the baseline(391).

Conclusions

Optimization Interactions

Refer to our final design results graphs below this paragraph. We combined these two optimizations, where setting aspect ratio to 1.09 and SRAM number of words to 1024 plus replacing the alu adder, the branch target address calculation, and unsigned multiplier of fast_multiply_div generally leads to strong improvements in performance. For instance, there is a decrease in critical path delay and an increase in worst slack. There is also a reduction in power both switching and internal, drc violations, and die area. Before, with any single optimization applied alone, there was no decrease in internal power, only switching; but now, with a combination of optimizations, internal power goes down. However, the negative impact is that total cell area increases over the baseline. The replacement with optimized adders and multipliers likely improved the timing for critical path delay and worst slack; both optimizations worked together to reduce power; potentially adjusting the aspect ratio and sram size increased cell area and core utilization. The final designs are located in the final_design folder for han carlson and koggestone respectively with both folders including the final gds file.



Summary

Summarize the improvements your team made to the Croc chip. This includes PPA improvements, functionality improvements, and verification/testability improvements. Do any of your improvements make the chip more expensive to manufacture? Why or why not? Do you think this expense is justified? Include a screenshot of your final floorplan (in OpenROAD or KLayout). 6-8 full sentences.

In this project, we made two main improvements to the Croc SoC. Firstly, we parametrized the aspect ratio and number of SRAM words by making relevant changes in the tcl scripts and rtl which lead to improvements in power and area. Secondly, we replaced the adder in the alu and

the addition in the branch target calculation and the unsigned multiplier in the core's fast multiply unit with optimized adders and multipliers generated from vlsiffra (with our implemented ihp13 tech support). This change led to improvements in critical path delay, worst slack, power, cell area, and drc violations. Combining these changes causes improvements in timing with smaller critical path delay, increased positive total slack, and massively decreased power both internal and switching, smaller die area, and fewer drc violations with the tradeoff of increased cell area. The tradeoffs with increased cell area does increase manufacturing costs, but this is balanced by changes like decreased power and die area which potentially decrease manufacturing costs. Whatever increased costs that arise are worth it for tangible timing improvements, reduced power, reduced die area.

